New infrastructures for Big Data: NoSQL Databases

VOLUME and VELOCITY

- A transaction represents the typical elementary unit of work of a Database Server
- A transaction identifies a unit of work performed by an application
- We also have VARIETY and VERACITY

Transactions

- The classical DBMSs (also distributed) are transactional systems: they provide a mechanism for the definition and execution of transactions
- In the execution of a transaction the ACID properties must be guaranteed
- New DBMS have been proposed that are not transactional systems

ACID

- Atomicity: A transaction is an indivisible unit of execution
- Consistency: the execution of a transaction must not violate the integrity constraints defined on the database
- Isolation: the execution of a transaction is not affected by the execution of other concurrent transactions
- Persistence (Durability): The effects of a successful transaction must be permanent

NoSQL databases

- It has been realized that it is not always necessary that a system for data management guarantees all transactional characteristics
- The non-transactional DBMSs are commonly called NoSQL DBMSs
- This really is not correct, because the fact that a system is relational (and uses the SQL language) and that it has transactional characteristics are independent

BIG DATA and the Cloud

DATA CLOUDS:

ON DEMAND STORAGE SERVICES, reliable, offered on the Internet with easy access to a virtually infinite number of storage resources, computing and network

CLASSIFICATION OF POSSIBLE METHODS OF STORAGE:

- Centralized or distributed, transactional, based on a traditional model (eg relational) remain the most widely used for traditional business applications (business information systems etc.).
- Federated and multi-databases, generally for companies and organizations that are associated and share their data on the Internet
- Cloud databases, to support BIG DATA by means of load sharing and data partitioning

NoSQL databases

- Provide flexible schemas
- The updates are performed asynchronously (no explicit support for concurrency)
- Potential inconsistencies in the data must be solved directly by users
- Scalability: no joins, ease of clustering, no 2PhaseCommit
- Evolution to a “simpler” schema: key/value-based, semi/non-structured
- Object-oriented friendly
- Caching easier (often embedded)
- Easily evolved to live replicas and node addition, made possible by the simplicity of repartitioning of data
- DO NOT support all the ACID properties

DATA MODEL

3/4 categories:

- Key–Value
- Document–based
- Column–family
- Graph-based

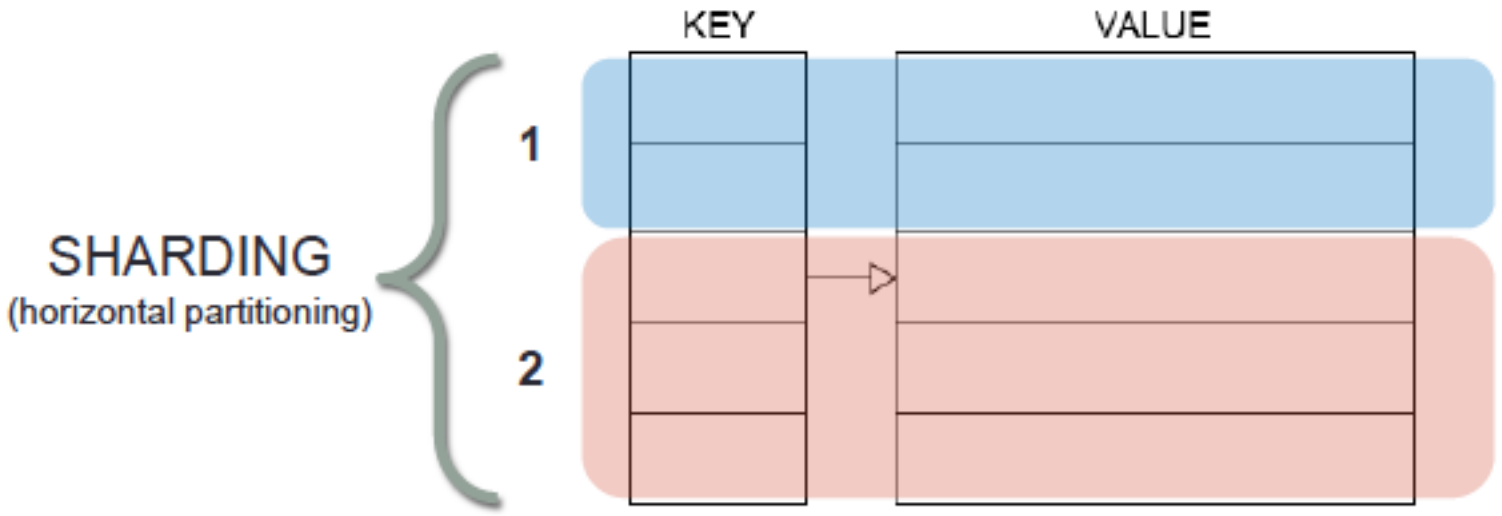
... Graphs not treated here, a separate evolutionary path from the other categories. They rely heavily on modeling relationships

Key -Value

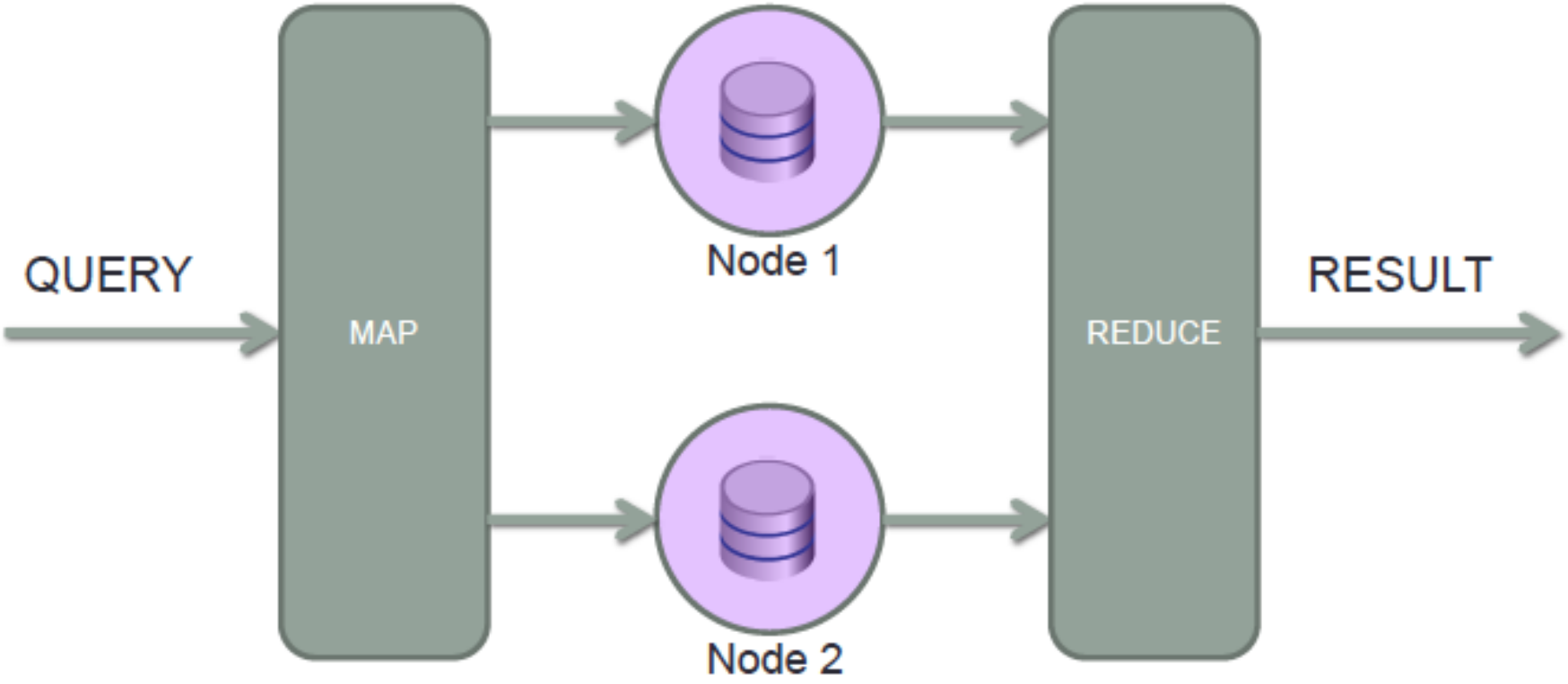
- Classical reference model of NoSQL systems
- Key: single or compound
- Value: blob, " opaque”
- Querying = find by key
- No schema (a dictionary)
- Standard APIs: get / put / delete

Key-Value: Scaling on multiple nodes

- Joins limit scalability
- No relationships in the database → easier to scale!
- Decoupled and denormalized entities are ‘self-contained'
- We can move them to different machines without having to worry about “neighborhood”!
- Sharding (horizontal partitioning)



Map Reduce



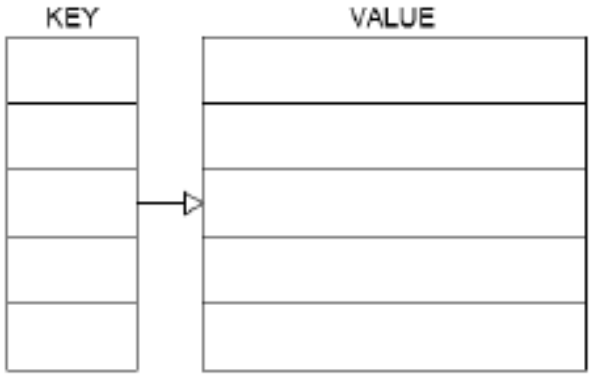
@Oscar Locatelli

Map-reduce paradigm

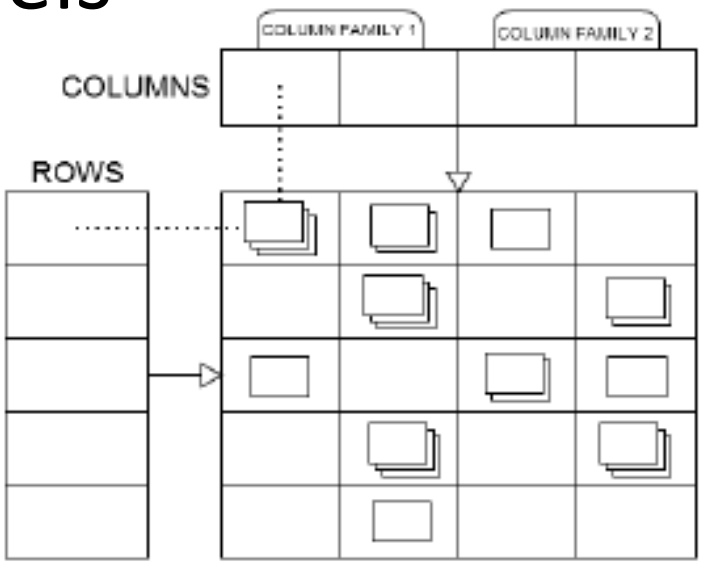
- Apache Hadoop was the first high-impact proposal
  - Hadoop 1.0 only had HDFS and MapReduce
- Hadoop 2.0 is more flexible
  - YARN, for resource description and management
  - Pig, Hive, Spark, Kafka (message broker), Sqoop (from DBMSs to Hadoop)
- Apache Spark is currently receiving a lot of attention
  - MapReduce stores in mass memory intermediate results, with high costs when executing multiple steps
  - Spark manages a direct transfer from one step to the next, similar to pipelines in DBMS query plans
- Several components. Among them:
  - MLlib (machine learning)
  - SparkSQL

Further Models

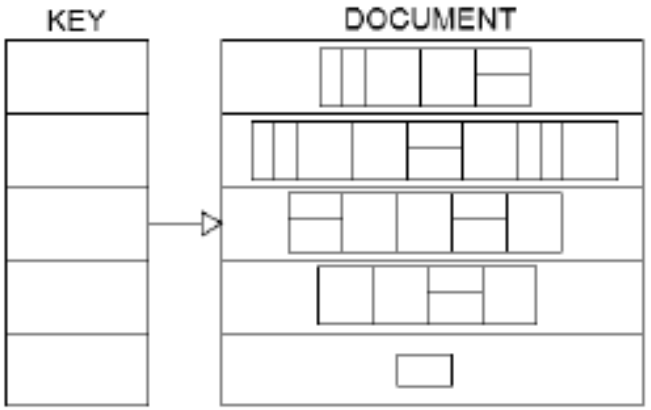
Key-Value



Column oriented



Document based



Column -Family 1/2

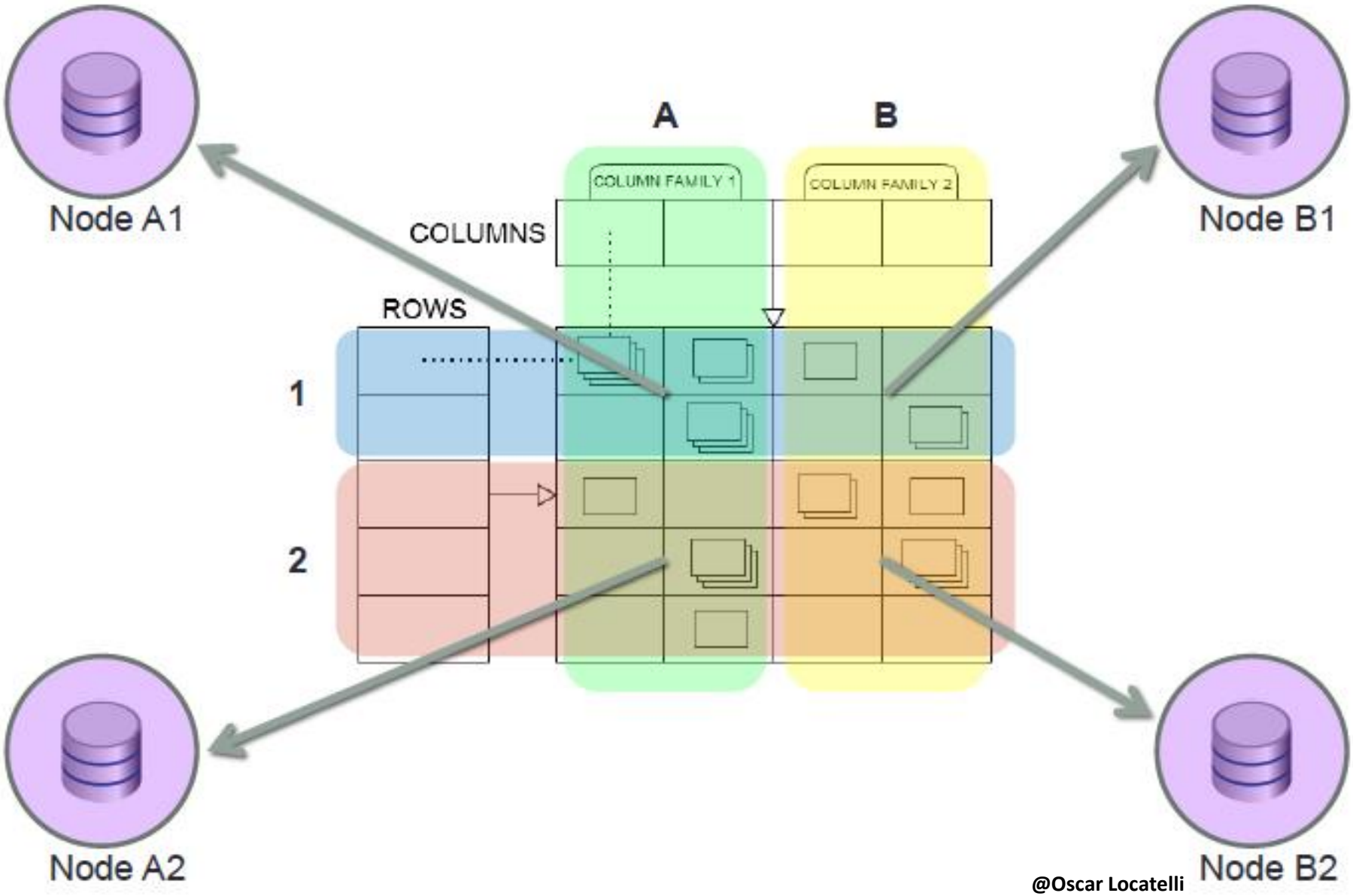
- Key is a triple row / column / timestamp
- Value: "opaque”
- Strongly oriented to Big Data, maximum scalability, the data is partitioned horizontally (sharding) and vertically (different columns on different machines) based on the keys of the row and column
- Ability to query: get or filter only on row and column keys (indexed). Supports projection
- Sorting is automatic, based on the names of rows and columns, so schema design is very important
- Standard APIs: get / put / scan / delete

Column–Family 2/2

Semi-structured diagram:

- Row: columns indexed within each row by a row-key
- Column-family: a set of columns, normally similar in structure to optimize compaction.
- Columns in the same column family will be “close”: to be determined at design time. Equivalent to tables in the RDBMS, but are somehow “semi-structured” (empty column-families occupy 0 bytes)
- Columns: have a name, indexed (column-key) and may contain a value (multi-version) for each row. Can be dynamically added (empty column occupies 0 bytes)

Column-family: maximum scaling



Document-based

- Key
- Value: collection of documents whose structure is not fixed and not even comparable to XML or JSON, sometimes “nested”
- Several mechanisms of the RDBMS, such as indexes, views, triggers, transactions (but very different in semantics)
- Ability to query: very rich: filters (and also indexes) on the internal fields of the document
- Schema-free and less ’sensitive' to design than a column-based
- API: high–level, conceptual (like ORM)

The CAP theorem

A data management system shared over the network (cloud, networked shared-data system) can guarantee at most two of the following properties:

- Consistency (C): all nodes see the same data at the same time
- Availability (A): a guarantee that every request receives a response about whether it was successful or failed
- (tolerance to) Partitions (P): the system continues to operate despite arbitrary message loss or failure of part of the system

According to Brewer's 2012 article, "2 of 3" is misleading:

  - because partitions are rare, there is little reason to forfeit C or A when the system is not partitioned.
  - the choice between C and A can occur many times within the same system at very fine granularity; not only can subsystems make different choices, but the choice can change according to the operation or even the specific data or user involved.
  - all three properties are more "fuzzy" than binary

The CAP theorem

- In ACID, the C means that a transaction preserves all the database constraints
- the C in CAP refers only to copy consistency, a strict subset of ACID consistency.

From this we can understand why in more traditional applications, such as banking or accounting, or bookings etc. these systems can be catastrophic

Interesting applications:

- data collected from sensors (append-only)
- In general, datasets that are seldom updated

SOME FAMOUS IMPLEMENTATIONS

- Amazon DynamoDB
  - Key–value
  - CAP : AP - guarantees Availability and Partition tolerance, relaxing Consistency
  - auto – sharding
  - P2P networks
  - It aims to eliminate the job of the database administrator
  - Project Voldemort, SimpleDB
- Google BigTable
  - Column- oriented, on the Google BigTable paper serves as the foundation to the NoSQL Column- based data –model
  - CAP: CP - if there is a network partition Availability is lost, but ‘strict' consistency may be required
  - auto-sharding, conflict resolution manual, no P2P

SOME FAMOUS IMPLEMENTATIONS (II)

- Hypertable and Hbase
  - Implementations of BigTable (built on Google File System)
  - Both within Apache Hadoop (framework for distributed applications based on map-reduce)
  - Interface Thrift, REST and APIs for various languages
  - HBase extensible (coprocessors), Hypertable most powerful
- Cassandra
  - Free from the Apache Foundation, Unix-like and Windows
  - Super-Column family
  - CAP : AP consistency with configurable auto - sharding, automatic conflict resolution
  - Combines the P2P with the data-model of BigTable
  - Transactions lock-free
  - Key names only on rows and columns
  - Also on Hadoop

SOME FAMOUS IMPLEMENTATIONS (III)

- Redis
  - Originally developed by Salvatore Sanfilippo
  - Key-value model
  - Supports storage in mass-memory, but it operates in main memory
  - High performance and scalability, often used as a cache backend for Websites
  - Some limited support is offered for joins, but it has a big impact on performance, as it runs scripts on the server

SOME FAMOUS IMPLEMENTATIONS (IV)

- MongoDB
  - Embeddable only in a C++ process, with LGPL license
  - Document–based
  - CAP: CP
  - auto-sharding with configurable strategy
  - static and automatic Indices created synchronously with the write
  - No transactions but atomic operations, and patterns for creating 2-phase commit or other policies
  - Query Task executed in Map-Reduce or otherwise distributed systems
  - In-place update of document attributes
  - Optimized for write-heavy (updates on index update and maintain consistency)
  - APIs for various languages ORM-like

  → It is the most widely used and known, excellent performance and excellent documentation

    → But, it is not a DBMS

SOME FAMOUS IMPLEMENTATIONS (IV)

- CouchDB
  - Document
  - CAP: AP, Multi-Version Concurrency Control, strict consistency of master, slave where appropriate
  - View materialized on the first read and updated with map-reduce algorithm written in Javascript, projection, sort and calculations
  - Transactions lock free
  - Write-heavy, Read-heavy
  - CouchDB is also a WebServer, can do application hosting HTML5 + JavaScript that are treated as documents (then synchronized between multiple databases, easy load-balancing)
  - Couchbase, CouchDB offers Memcached + GeoIndex
  - CouchDB is the basis of the synchronization service Ubuntu One

Different products for different objectives

- Enterprise Big Data: maximum performance and scalability: Amazon DynamoDB, Google BigTable and all their derivatives (Voldemort, Cassandra, HBase, Hypertable)
- Document Management or RDBMS replacement (with extreme caution!!!): MongoDB (write heavy) RavenDB (read-heavy), BigCouch (both not discussed here)
- Scenarios with online-offline, mobile devices/ laptops: CouchDB (Ubuntu One)

Other interesting solutions

- CockroachDB
  - Emphasizes reliability, scalability and distribution
  - It is serverless
  - It uses Raft, an evolution of Paxos commit
- Firebase
  - Developed by Google, well supported within GCP
  - It goes beyond classical DBMS tasks (it is presented as an app development platform: auth, log collection, …)
  - Firestore is now the NoSQL DBMS component
- Supabase
  - Open source alternative to Firebase
  - It starts from PostgreSQL
  - It offers scalability (sharding) and sophisticated access control, adapting triggers to the management of sophisticated rules

Bibliography on NoSQL

BOOK :

Tamer Ötzsu M., Valduriez P. – Principles of *Distributed Database Systems: 3rd ed.* - Springer, 2011

More references

•Abadi Daniel J. - *Data Management in the Cloud: Limitations and Opportunities* - IEEE Data Engineering Bulletin, Vol. 32 No. 1, March 2009

http://sites.computer.org/debull/A09mar/A09MAR-CD.pdf#page=5

•Dean J., Ghemawhat S. – *MapReduce: A Flexible Data Processing Tool* - CACM, Vol.53, n. 1, pp. 72-77, 2010

•Foster I., Yong Zhao, Raicu I., Lu S - *Cloud Computing and Grid Computing 360-Degree Compared* - Grid Computing Environments Workshop 2008, pp. 1-10, 2008

http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4738445